OOP in C++

Introduction

Instructor: Dinesh Maharjan

2022, November

Computer Program

- A program is a set of instructions following the rules of the chosen language.
- Without programs, computers are useless.
- A program is like a recipe.
- It contains a list of ingredients (called variables) and a list of directions (called statements) that tell the computer what to do with the variables.

Programming Language

- A vocabulary and set of grammatical rules (syntax) for instructing a computer to perform specific tasks.
- Programming languages can be used to create computer programs.
- The term programming language usually refers to high-level languages, such as BASIC, C, C++, COBOL, FORTRAN, Ada, and Pascal.

Programming Language

- You eventually need to convert your program into machine language so that the computer can understand it.
- There are two ways to do this:
  - Compile the program
  - Interpret the program

Programming Language

- **Compile** is to transform a program written in a high-level programming language from source code into object code.
- This can be done by using a tool called **compiler**.
- A compiler reads the whole source code and translates it into a complete machine code program to perform the required tasks which is output as a new file.

Programming Language

- **Interpreter** is a program that executes instructions written in a high-level language.
- An interpreter reads the source code one instruction or line at a time, converts this line into machine code and executes it.

Computer Programming

- Computer programming is the process of writing, testing, debugging/troubleshooting, and maintaining the source code of computer programs.
- This source code is written in a programming language like C++, JAVA, Perl etc.

Computer Programmer

- A **programmer** is someone who writes computer program.
- Computer programmers write, test, and maintain programs or software that tell the computer what to do.

Assembly Language

- Example:
  - Machine language :

    10110000 01100001
  - Assembly language :

    mov a1, #061h
  - Meaning:

  Move the hexadecimal value 61 (97 decimal) into the processor register named "a1".

What is a programming paradigm?

- A classification of programming languages based on their features (but most popular languages support multiple paradigms)
- A programming style or way programming/thinking
- Example: Object Oriented Programming (thinking in terms of objects which contain data and code)
- Many common languages support (to some extent) multiple paradigms (C++, python, ...)

Why should I care?

- Discover new ways of thinking $\longrightarrow$ challenge your current believes about how to code
- Choose the right paradigm for the right problem or pick the best of many worlds

Programming Paradigms

- Imperative
  - Procedural
  - Structured/Object-oriented
- Declarative
  - Functional
  - Logic

Imperative Programming

- It clearly specifies what to do and how to do.
- Closer to structure of machine
- Program is sequence of instructions
- Order of instruction is important.
- Use mutable and immutable data.
- Have state and can be changed
- Instructions can modify the memory state by moving bits.
- E.g. Algol, Fortran, Cobol, Pascal, C, Basic, Ada
- Types are Structured Programming, Procedural Programming, modular programming, OOP

Declarative Programming

- In this paradigm we can specifies what to do in form of logic of computations
- We do not specify how to do computations.
- The program will itself do computations for us.
- Instructions do not modify the memory state.
- Rather it creates new variables.
- e.g. Haskell, Prolog
- Types are Functional and Logic Programming

Imperative vs. Declarative

- Imperative programming uses a state-based model of computation (**Turing machine**); expresses programs in terms of sequences of command statements to change states
- Declarative programming uses a function-based model of computation (**Lambda calculus**); expresses programs as logical or functional statements, without *control flow*

Functional programming

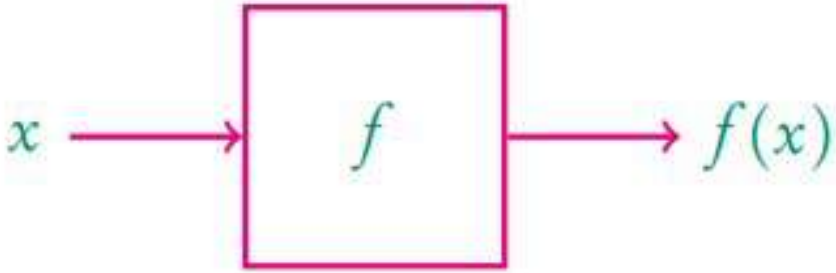
- LISP, Scheme, Haskell, SQL, Lex/Yacc
- Computation as evaluation of mathematical functions; implementation left to compiler
- As opposed to 'functions' in procedural languages: no side effects, *referential transparency*
- Used more in academia, not so much in commercial or industrial applications

Functional Programming

- Uses functions, expressions and recursion.
- Fixed output for fixed input
- No side effects. Does not change outside world.
- No mutable state
- Fewer bugs
- Lazy evaluation
- Requires huge memory

Programs as functions

- Functions transform inputs to outputs



- **Program**: rules to produces outputs from inputs
- **Computation**: process of applying the rules

Defining functions (2)

- Not the same as:

```
add :: (Int, Int) -> Int
add (x, y) = x + y
```

- This `add` is a function of one argument, which is a tuple of two `Int`s
  - still returns an `Int`

Laziness (3)

- Example:

```
let f x = 10
f (1/0)
```

- In strict language, this causes an error
- In lazy language this returns `10`
  - `1/0` is never evaluated, because it wasn't needed
- Big deal, right?

Logic Programming

- Programs are written in the form of facts and rules.
- These facts and rules are used to generate conclusions or new facts.
- User can type series of questions or queries.
- These queries are evaluated true or false based upon the facts and rules in the program.
- e.g. Prolog

Logic Programming

```
dog(fido).
dog(rover).
dog(henry).
cat(felix).
cat(michael).
cat(jane).
animal(X):-dog(X).
```

Logic Programming

?-dog(jane). **no**	[Is jane a dog? No - a cat]
?- animal(fido). **yes**	[Is fido an animal?] [yes - because it is a dog and any dog is an animal]
?- dog(X). **X = fido ;** **X = rover ;** **X = henry**	[Is it possible to find anything, let us call it X, that is a dog?] [All 3 possible answers are provided]

Logic Programming

```
happy(yolanda).
listens2music(mia).
listens2music(yolanda):- happy(yolanda).
playsAirGuitar(mia):- listens2music(mia).
playsAirGuitar(yolanda):- listens2music(yolanda).
```

```
?- playsAirGuitar(mia).
yes
?- playsAirGuitar(yolanda).
yes
```

Logic Programming

- Prolog, Datalog
- Theory of computation based on first-order logic
- Typically uses *Horn clauses* to make declarative statements:

  grandparent(A,B) if **parent**(A,C) and **parent**(C,B)

- Can be seen procedurally as *goal reduction*

Procedural programming

- C, C++, Fortran, Pascal, BASIC
- Break down your task into variables, data structures and subroutines
- Use of procedures, modularity for efficiency and clarity (e.g., *scoping*)
- Allows for development of shared libraries

Structured programming, OOP

- Structured: Extensive use of subroutines, blocks, for/while loops (as opposed to goto); modularity very important
- OOP (Smalltalk, VB.NET, C#, Java, Python, Ruby): Arrange data attributes and methods into *objects*; break down your programming task into a collection of interacting classes of objects
- Control flow less clear in OOP; in this sense less 'imperative'

Structured Programming

- It is top down approach of programming.
- Program is created as set of functions.
- It decomposes whole program into simple atomic functions
- The problem is solved by these function.
- They share common data called global data.
- Result of processing by these function is stored in global data.

Structured Programming

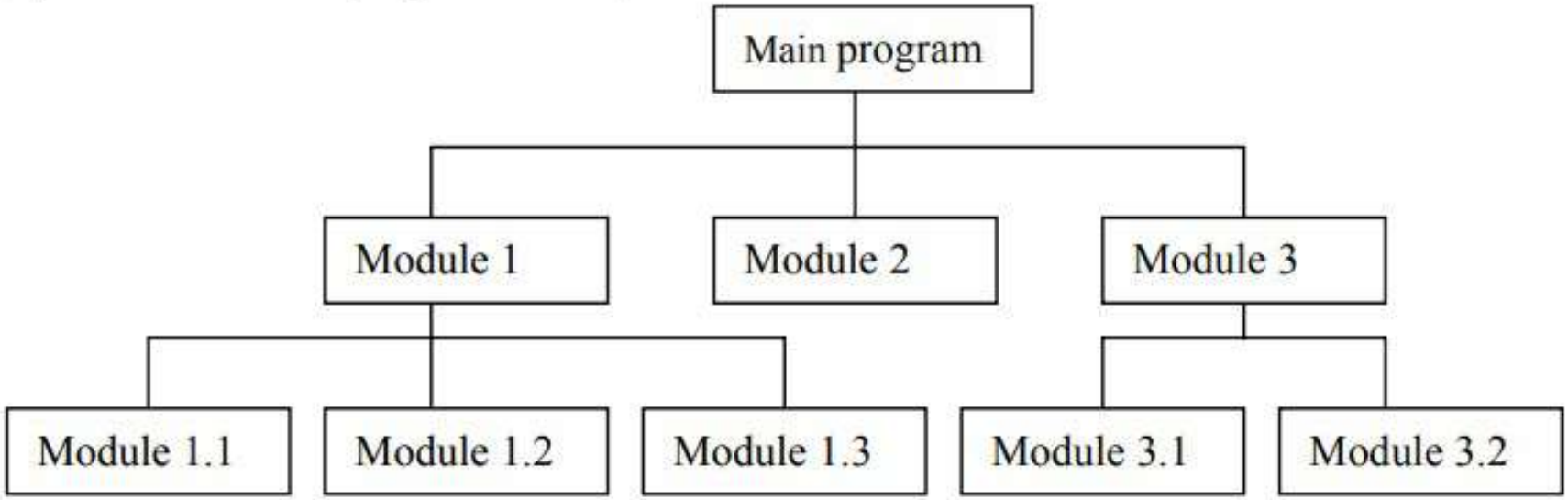


Fig. 2 Structure chart

Object Oriented Programming

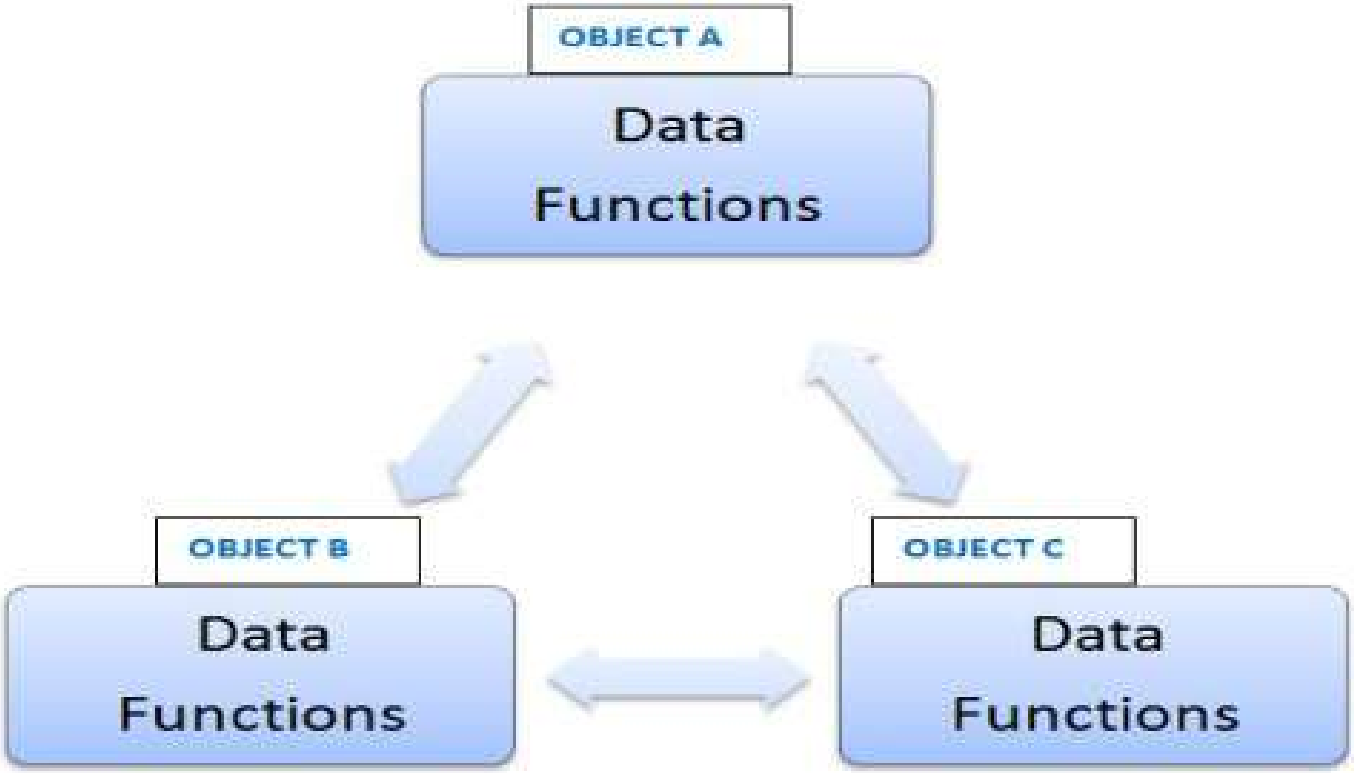
- In real world, everything we see are objects.
- We use objects to solve different type of problems
- For eg, Car object is used to travel from one place to another.
- Camera is used to take pictures.
- House is object where stay.
- Each object have their own properties and behaviour.
- OOP can model real world objects

Object Oriented Programming

- OOP is bottom up programming approach.
- Program is organized as set of objects.
- Problem is solved by communicating among these objects.
- We start from related specific problem.
- And combine these related problem to solve general problems.
- It helps to reuse the existing modules.
- OOP is data centric.
- OOP designs type first and then come up with methods to work on these types.

Object Oriented Programming

Object Oriented Programming



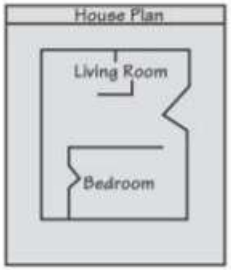
Class

- We model real world objects using class.
- We determine real world objects behaviour and properties.
- Then define class including those behaviour and properties
- So, class is description or blue print of objects
- Contains attributes (properties, characteristics) of object
- Contains methods (functions, behaviour) of object

Classes and Objects

- A Class is like a blueprint and objects are like houses built from the blueprint

Blueprint that describes a house.



Instances of the house described by the blueprint.

Class Design

Car Class:

Properties:
Brand, maxSpeed, engineCapacity

Behaviour:
Start(), accelerate(), break(), changeGear()

Printer Class:

Properties:
Brand, storageSize, resolution

Behaviour:
click(), display(), delete()

Object

- In real world, anything that have some identity, characteristics and behaviour can be taken as object
- We create objects from class.
- Defining class does not allocate memory.
- When objects are created, memory is allocated.
- The real house developed from blue print.

Abstraction

- OOP provides abstraction by hiding unnecessary details from the users.
- For eg. with phone we can make calls.
- User must not know how phone is making connection and how voice is being transferred.
- User must know how to make call.
- Same thing applies with car object.

Abstraction

- Abstraction is achieved by defining methods inside class
- Methods can access data associated with objects
- Users must not know the details of these methods.
- However, user must know how to use those methods and their results.
- OOP defines class: a type of abstract data type data has hidden data and methods for different actions.

Data hiding

- OOP hides data to protect it from unauthorized access.
- Accessing data is restricted to associated methods.
- Unlike structured programming, where global data by unrelated users, OOP helps to prevent this.
- Data can be hidden by specifying different types of access specifiers such as private, protected.

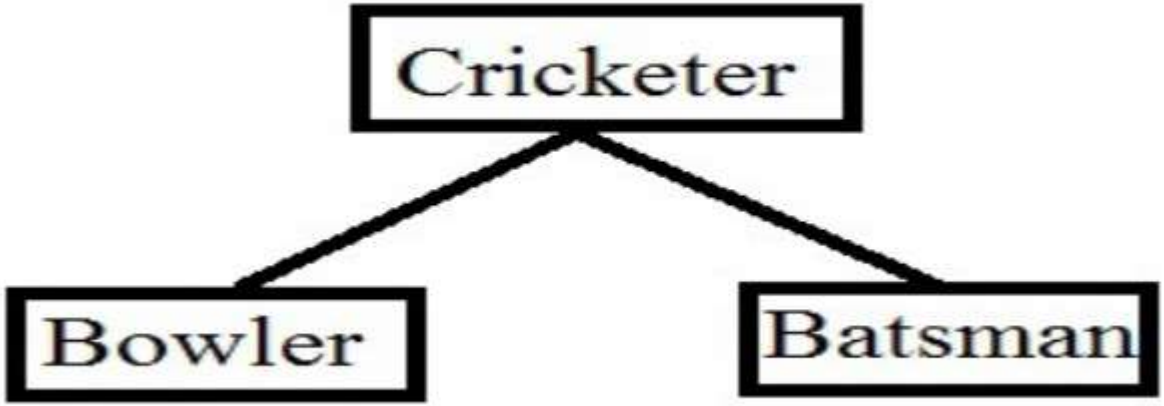
Data Encapsulation

- OOP hides data from other objects.
- So, there must be some mechanism to access these data.
- OOP allows to access hidden data through member function.
- So, keeping hidden data and methods to access them in the same class is called encapsulation.
- OOP defines getter and setter methods to access and initialize hidden data.

Inheritance

- OOP exploits similarities between classes by creating new classes from exiting ones.
- The new class adds additional properties and behaviour.
- This helps to reuse the classes and boost the application development.
- This also helps to fast debug.
- The new class is called derived class and existing class is called base class.

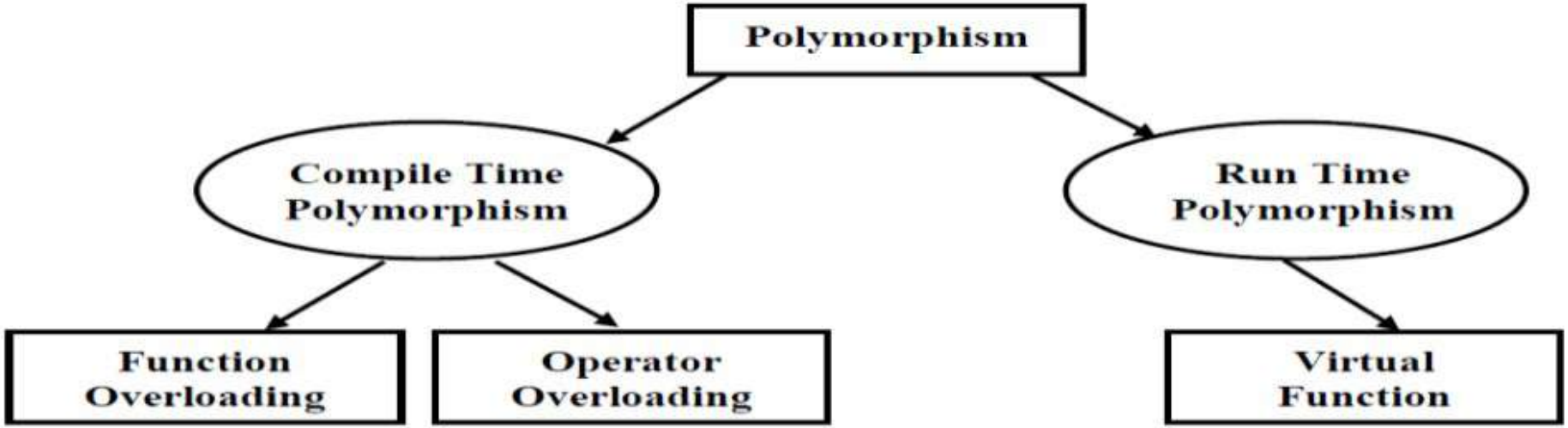
Inheritance



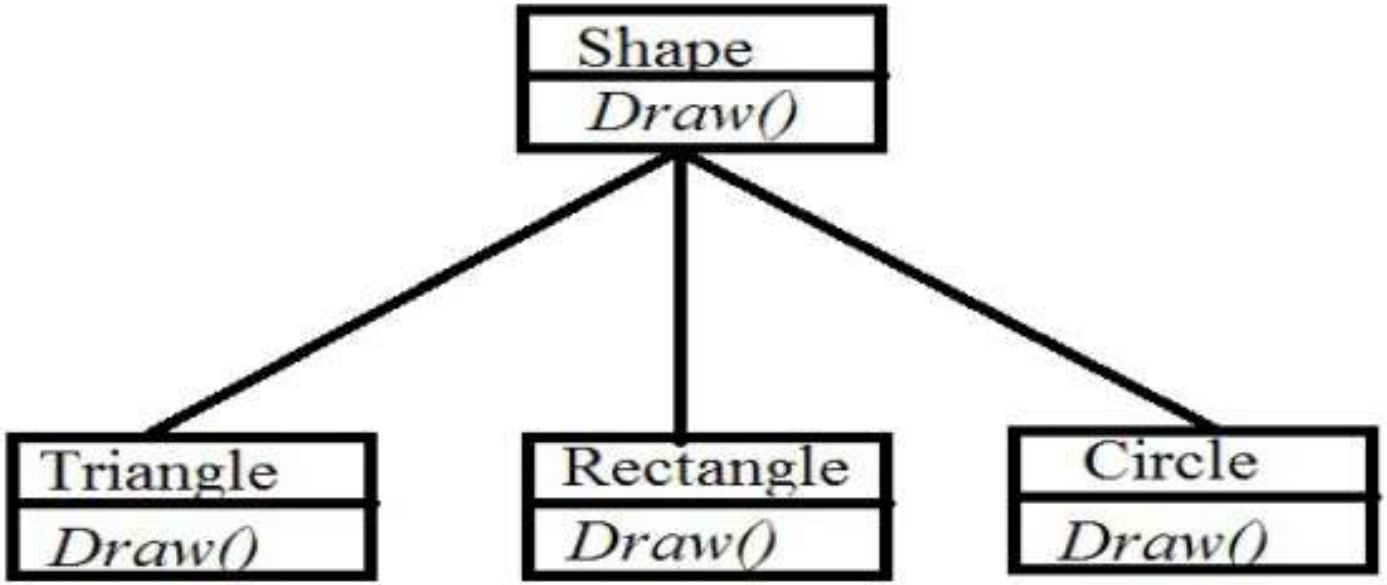
Polymorphism

- In polymorphism, poly means multiple and morphism means state.
- OOP allows to create methods with same name but different parameters and different actions.
- Polymorphism is achieved through two ways.
- Overloading
- Method overloading and operator overloading
- Overriding
- Method overriding

Polymorphism



Function Overloading



OOP VS Structured

Procedure Oriented programming	Object oriented Programming
i)Focus is on procedure rather than data.	i)Focus is on the data rather than Procedure.
ii)Data is not secure .	ii)Data is secured.
iii)Employs top-down approach.	iii)Employs bottom-up approach.
iv)It does not model real world's objects.	iv)It model the real world's objects.
v)Programs are decomposed into functions	v)Programs are decomposed into objects.